

Update 2

Swarm Search -- Development Progress

What changed this round, and how the swarm knows where obstacles are -- including why a learned policy can search unknown terrain without crashing

What This Update Covers

This update covers the work done since the first development report. The headline changes: the swarm now moves intelligently instead of randomly, each drone tracks battery, and the whole simulation can be watched and controlled live on a real interactive map in a browser -- including picking which real area to search and how many drones to send.

Changes at a glance

- New movement intelligence: replaced random motion first with a potential-field rule, then with a BFS shortest-path coverage policy that roughly doubled area coverage and can't get stuck.
- Battery tracking: every drone now has a real, draining battery, shown live next to it.
- Live web map: a browser-based view (FastAPI + WebSocket + Leaflet, no paid API key) that animates the swarm over real streets and buildings in real time.
- Mission control: a sidebar to pick the search site (San Jose, Berkeley, Golden Gate Park) and set the swarm size (1-10 drones), then redeploy live -- radar-style.
- A written development report PDF documenting the full system.

The rest of this document walks through each change, then gives a detailed, honest answer to the key question: how does the swarm know where obstacles are, and why would a trained policy avoid crashing in terrain it has never seen before?

Change 1: Movement Intelligence

Previously every drone moved completely at random, ignoring its own sensors. This round added real decision-making, in two stages.

Stage 1 -- Potential fields (a classic robotics rule)

Each drone sums three 'forces': push away from nearby obstacles, push away from nearby teammates (so the swarm spreads out), and pull toward uncovered ground. It then moves in the resulting direction. This works, but has a well-known weakness: the forces can cancel out and trap a drone oscillating between two cells. Several fixes helped but the problem is structural to the approach, so it was replaced.

Stage 2 -- BFS shortest-path coverage (current default)

Each drone runs a breadth-first search over open cells to find the real shortest path to the nearest unsearched cell it can reach, and steps along it. Drones claim different target cells so they naturally divide up the area. Because progress is measured in whole grid-steps toward a target, a drone literally cannot loop forever the way the force-based version could.

Measured result

Averaged over 15 randomized test maps, mean area coverage per mission:

- Random movement: about 26%
- Potential fields: about 38%
- BFS coverage: about 77% -- roughly double the potential-field version, and confirmed at about 79% on the real San Jose map.

Changes 2-4: Battery, Web Map, Controls

Change 2 -- Per-drone battery

Every drone starts a mission at 100% battery. Moving costs more per step than hovering (mirroring real drone motors drawing more power than idling), and the level is shown live next to each drone, turning red when low. Battery is real simulation state, not just a label -- it is exactly the kind of small status a real drone would broadcast to the team over radio.

Change 3 -- Live web map

A new backend runs the simulation continuously and streams each step to the browser over a WebSocket. The browser draws it on a real Leaflet map (OpenStreetMap tiles -- free, no API key or billing setup). Real buildings appear as dark obstacles, searched ground fills in green, and each drone is a small marker that rotates to face its heading. A sidebar shows step count, overall coverage, and a battery bar per drone.

This was deliberately built on OpenStreetMap/Leaflet rather than the Google Maps API specifically to avoid requiring a billing-enabled API key for a project at this stage -- the visual result (real streets and buildings) is equivalent for the demo's purpose.

Change 4 -- Mission control (pick a zone, send N drones)

The sidebar has a search-site dropdown (Downtown San Jose, UC Berkeley, Golden Gate Park), a slider for swarm size (1-10 drones), and a 'Deploy swarm' button. Pressing deploy rebuilds the mission for that real area with that many drones and every connected browser updates instantly. Note: these are real places used as stand-in search areas, not a live disaster feed -- there is no real-time incident data source wired in.

How Obstacles Are Known -- Today

This is the most important thing to be precise about, because the honest answer is different today than it will be after adding a learned policy, and confusing the two would overstate what currently exists.

Today: the map is known in advance

Right now the obstacles come from OpenStreetMap -- real building, water, and forest outlines for the chosen area, downloaded and marked onto the grid before the mission starts. So the system is effectively handed an accurate map. Each drone's 'sensor view' is just a lookup into that known map within a small radius around it.

Because the map is known, avoiding obstacles is not really a perception problem at all -- it collapses into path planning around shapes we already know about. That is exactly what the BFS policy does: it only ever plans routes through cells already known to be open, so it never even considers stepping into a building. No sensing, no learning, and no crashing -- but only because the map was correct and complete to begin with.

Why that is not the real search-and-rescue problem

In an actual disaster, you cannot assume an accurate map. Buildings may have collapsed, roads may be blocked, and new hazards appear that no map shows. A real search drone has to discover obstacles live from its own sensors, not look them up. The known-map version built so far is the right foundation and a strong baseline, but it is the easier half of the problem.

The distinction in one line

Known map -> plan around obstacles you were told about (done today). Unknown terrain -> discover obstacles as you go and never hit one you haven't seen yet (the next milestone, explained on the following pages).

Unknown Terrain: Sensing Obstacles Live

In the unknown-terrain version, no map is given. Each drone builds its own picture of the world as it flies, from onboard sensing. Mechanically, this works through two standard robotics ideas.

1. Local sensing (range sensing / ray casting)

The drone can only 'see' a limited distance around itself -- like a camera or lidar with finite range. At each moment it detects which nearby cells are blocked and which are open, but only within that sensor range. Anything beyond it is genuinely unknown until the drone gets closer.

2. An occupancy grid (the drone's live memory of the world)

As the drone moves, it records what it has sensed into its own map, where every cell is one of three states: known-free, known-blocked, or not-yet-seen. This map starts almost entirely 'unknown' and fills in as the swarm explores. Teammates can share pieces of their maps so the swarm builds a shared picture faster than any single drone could alone.

Why this prevents crashing, even with no prior map

The key safety rule is simple: a drone is only ever allowed to move into a cell it has already sensed to be free. It never moves into 'unknown' space blindly -- it moves to the edge of what it has confirmed is open, sensing further as it goes. An obstacle is detected before the drone reaches it, because sensing range extends past one step of movement. So even in terrain it has never encountered, the drone does not crash into a newly discovered obstacle: it sees the obstacle first, marks that cell blocked, and simply plans around it. This is true whether the drone is controlled by a hand-coded rule or a trained policy.

Why a Learned Policy Handles Unknown Terrain

A fair question: if the drone is controlled by a trained neural-network policy, why would it behave sensibly in a specific real place it was never trained on -- why wouldn't it crash there? The answer comes down to WHAT the policy actually learns from.

The policy never sees the whole map -- only a local view

By design, a drone's input is only its small local sensor patch (what is blocked/free/unsearched right around it) plus its own status. It is never given the identity of the map or a god's-eye view. So the policy physically cannot learn 'in San Jose, turn left at this building' -- that information is not in its input. All it can learn is a general reaction: 'given this local pattern of obstacles and open space around me, move this way.'

Training on many random maps forces a general skill, not memorization

This is exactly why the environment generates a different random obstacle layout every training episode. If the drone trained on one fixed map, it could memorize that map. Training across thousands of different random layouts makes memorizing impossible -- the only thing that reliably earns reward across all of them is learning the general skill of reading local surroundings and reacting safely. A brand-new real place is then just another never-before-seen layout, which is precisely what the policy was trained to handle.

Safety does not depend on the policy being perfect

Even a trained policy can be paired with the same hard safety rule from the previous page: the environment simply never lets a drone move into a cell known to be blocked (blocked moves are rejected). So the learned policy is responsible for searching efficiently, while the 'never enter a sensed obstacle' guarantee is enforced structurally underneath it. The policy learns good strategy; it is not solely trusted to avoid collisions.

Analogy, Status, and Next Steps

A short analogy

A trained policy is like a person who has explored thousands of different buildings blindfolded using only a cane. Drop them into a building they have never entered and they still won't walk into walls -- not because they memorized this building, but because they learned the general skill of feeling ahead and reacting. They tap first, then step. The drone senses first, then moves.

Honest status: what is built vs. planned

- Built and tested: known-map version (real OSM obstacles), BFS coverage policy, battery, live controllable web map. In this version obstacle avoidance is solved by planning around a known map.
- Planned next: the unknown-terrain version -- local sensing plus a live occupancy grid, so obstacles are discovered rather than looked up; and a trained policy evaluated against the BFS baseline under those conditions.

In other words, the safety argument on the previous pages describes how the unknown-terrain version will work by construction. The 'sense before you step' rule is what makes not-crashing a guarantee rather than a hope, and it is deliberately kept independent of whether the pilot is a hand-coded rule or a learned policy.

Suggested build order

- 1. Add local sensing + occupancy grid (turns lookup into discovery).
- 2. Add realistic limited communication between drones.
- 3. Train a policy and compare it to the BFS baseline under those unknown-terrain, limited-communication conditions.